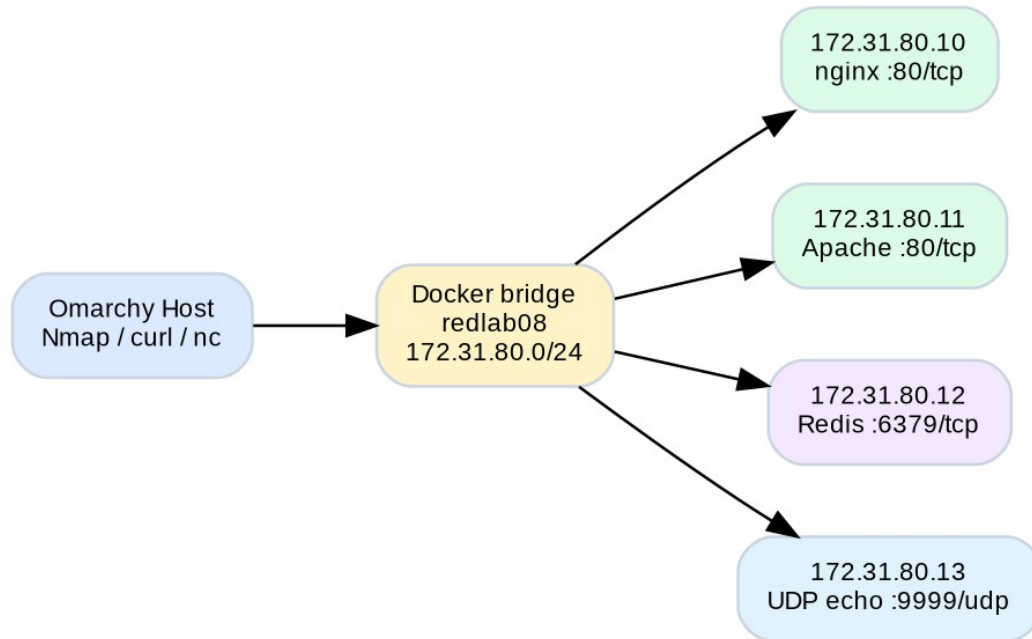


RED TEAM LEARNING PATH

MODULE 08

Network Scanning & Enumeration

Omarchy / Arch Linux Edition

*Training topology used throughout this module.***LEGAL & SAFETY BOUNDARY**

Semua scan pada hands-on module ini dibatasi ke loopback dan subnet Docker 172.31.80.0/24 yang dibuat sendiri. Jangan pindahkan command mentah ke jaringan publik, kantor, kampus, Wi-Fi tetangga, cloud account, atau target lain tanpa authorization tertulis dan scope yang jelas.

Version 1.0 | 04 Oktober 2026

1. Tujuan Pembelajaran

Setelah menyelesaikan module ini, lo harus bisa mengubah daftar target yang sudah authorized menjadi inventory host, port, service, dan bukti teknis yang dapat diverifikasi. Fokusnya bukan "scan sebanyak mungkin", tetapi membuat keputusan yang bisa dijelaskan dan diulang.

- [] Membedakan host discovery, port discovery, service detection, dan protocol-specific enumeration.
- [] Menjelaskan enam state port Nmap dan kenapa state bergantung pada sudut pandang scanner.
- [] Memilih antara TCP connect scan, SYN scan, dan UDP scan sesuai konteks.
- [] Menggunakan -sV, output XML, dan NSE secara hati-hati.
- [] Melakukan manual verification agar hasil scanner tidak diperlakukan sebagai kebenaran absolut.
- [] Menyimpan hasil scan sebagai evidence dan mengubah XML menjadi inventory terstruktur.
- [] Melakukan scanning hanya pada target yang sudah masuk scope.

2. Prerequisite & Tools

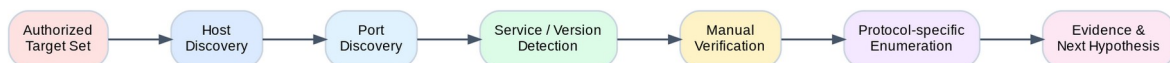
- Module 00-07 sudah dipahami, khususnya networking, HTTP, Linux, scope/RoE, dan recon.
- Docker berjalan di Omarchy.
- Tool utama: nmap, curl, openbsd-netcat, jq, Python 3, dan Docker.

```
omarchy pkg add nmap curl openbsd-netcat jq python
```

Omarchy note

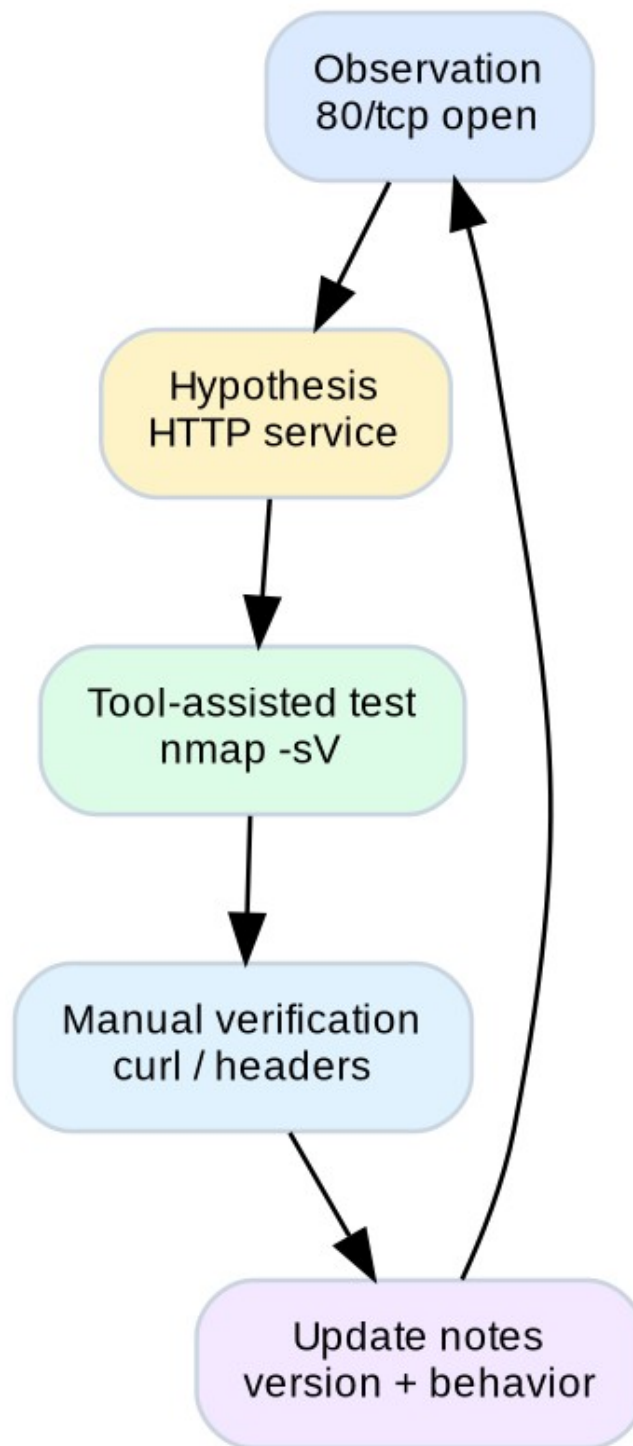
Package Nmap tersedia di repository resmi Arch. Gunakan omarchy pkg add untuk package biasa dan omarchy update untuk update sistem, konsisten dengan module sebelumnya.

3. Mental Model: Enumeration Bukan Satu Command



Enumeration pipeline: discovery sampai evidence dan next hypothesis.

Recon Module 07 menjawab "aset apa yang mungkin ada?". Module 08 menjawab "host mana yang reachable, port apa yang terlihat, service apa yang berjalan, dan apa bukti manualnya?". Enumeration adalah loop observasi -> hipotesis -> validasi -> dokumentasi.



Tool membantu pengujian; manual verification menjaga interpretasi tetap grounded.

4. Host Discovery

Sebelum memindai ribuan port, kita biasanya mempersempit target menjadi host yang hidup atau menarik. Nmap melakukan host discovery secara default sebelum port scan. Pada Ethernet lokal, ARP/Neighbor Discovery biasanya lebih efektif daripada ICMP saja.

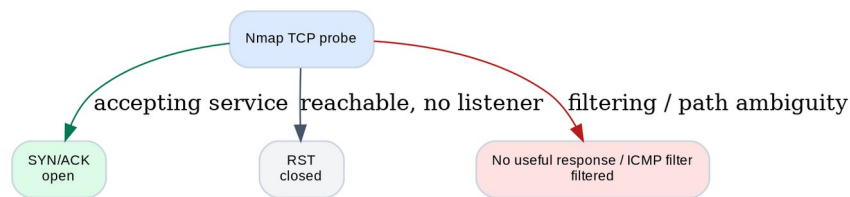
Command	Makna
<code>nmap -sn TARGET</code>	Host discovery saja, tanpa port scan.
<code>nmap -sL TARGET</code>	List scan; daftar target tanpa port scan, tetapi dapat melakukan DNS resolution.
<code>nmap -Pn TARGET</code>	Lewati discovery dan perlakukan target seolah online.
<code>nmap -n ...</code>	Lewati reverse DNS untuk mengurangi noise/latency.

Common mistake

`-Pn` bukan "stealth mode". Ia hanya melewati host discovery dan dapat membuat scan lebih lambat karena semua target diperlakukan online.

5. Port Scanning & Port States

Nmap tidak hanya mengenal open/closed. Ia menggunakan enam state: open, closed, filtered, unfiltered, open|filtered, dan closed|filtered. State tersebut menggambarkan apa yang dilihat scanner dari lokasi dan teknik tertentu, bukan sifat permanen sebuah port.



Simplified intuition for common TCP outcomes.

State	Interpretasi praktis
open	Ada aplikasi yang menerima koneksi/datagram.
closed	Host reachable, tetapi tidak ada listener pada port tersebut.
filtered	Filter/network path mencegah Nmap menentukan open vs closed.
unfiltered	Port reachable, tetapi teknik scan tertentu belum menentukan open/closed.
open filtered	Tidak cukup bukti untuk membedakan open dan filtered; umum pada UDP.
closed filtered	Tidak cukup bukti untuk membedakan closed dan filtered; muncul pada teknik tertentu.

6. TCP Connect vs SYN Scan

Scan	Contoh	Kapan berguna
TCP Connect	<code>nmap -sT -p 80 TARGET</code>	Tidak butuh raw packet privilege; menggunakan connect() OS.
SYN Scan	<code>sudo nmap -sS -p 80 TARGET</code>	Raw SYN probe; efisien dan umum saat privilege tersedia.

7. Memilih Port dengan Sengaja

Default Nmap memindai 1.000 port TCP yang dianggap paling umum. Untuk lab atau engagement, pilih port berdasarkan tujuan, bukan karena flag terlihat keren.

Pilihan	Contoh	Catatan
Specific ports	-p 22,80,443	Cepat dan hypothesis-driven.
Range	-p 1-1024	Cakupan eksplisit.
Top ports	--top-ports 100	Trade-off antara kecepatan dan coverage.
All TCP ports	-p-	1-65535; mahal dan hanya bila scope/waktu mendukung.
Open only output	--open	Menyederhanakan output, tetapi jangan kehilangan evidence bila audit butuh state lain.

Think like an attacker

Kalau recon menunjukkan web app pada custom port, scan "top 1000" saja bisa gagal menemukan service penting. Gunakan bukti recon untuk menentukan coverage, bukan hanya default tool.

8. Service & Version Detection

Nomor port hanyalah petunjuk. Service bisa berjalan pada port yang tidak biasa. Opsi -sV mengirim probe dan mencocokkan response dengan signature database untuk mengidentifikasi protocol, product, version, dan kadang CPE.

```
nmap -sV -p 80,6379 172.31.80.10
nmap -sV --version-light -p 80 172.31.80.10
```

Scanner output is evidence, not verdict

Banner/version detection bisa salah, diubah, diproxy, atau disembunyikan. Selalu cari konfirmasi manual sebelum menarik kesimpulan vulnerability.

9. UDP Enumeration

UDP tidak memiliki handshake seperti TCP. Banyak service UDP tidak merespons probe yang salah, firewall dapat membuang packet diam-diam, dan ICMP error sering dibatasi. Itu sebabnya UDP scan umumnya lebih lambat dan sering menghasilkan open|filtered.

```
sudo nmap -sU -p 9999 172.31.80.13
sudo nmap -sU -sV -p 9999 172.31.80.13
```

Protocol matters

DNS, SNMP, DHCP, NTP, dan banyak service lain memakai UDP. Mengabaikan UDP sepenuhnya dapat membuat inventory tidak lengkap.

10. Nmap Scripting Engine (NSE)

NSE menjalankan script Lua untuk discovery, version enrichment, authentication checks, vulnerability checks, bahkan kategori yang intrusive atau exploit. Script tidak dijalankan dalam sandbox. Karena itu, gunakan explicit script dan baca dokumentasinya sebelum eksekusi.

```
nmap --script-help http-title
nmap --script-help redis-info
```

```
nmap -p 80 --script http-title,http-headers 172.31.80.10
nmap -p 6379 --script redis-info 172.31.80.12
```

Peringatan -sC

-sC sama dengan --script=default. Dokumentasi Nmap menyatakan sebagian default scripts dianggap intrusive. Di module ini, -sC hanya boleh dicoba pada lab lokal yang lo bangun sendiri. Untuk engagement nyata, pilih script eksplisit berdasarkan RoE.

11. Output & Evidence

Jangan hanya copy-paste terminal. Nmap dapat menyimpan beberapa format sekaligus dengan -oA. Format XML paling berguna untuk automation karena memiliki struktur yang stabil.

```
mkdir -p module08-enum/02-scans
nmap -sV -p 80 -oA module08-enum/02-scans/nginx 172.31.80.10
```

- .nmap - human-readable normal output.
- .gnmap - legacy grepable output; praktis tapi tidak sekaya XML.
- .xml - paling cocok untuk parser, inventory, dan tooling berikutnya.

Evidence integrity

Simpan command, timestamp, target, dan hasil mentah. Jangan hanya menyimpan screenshot. Screenshot bagus untuk report, tetapi raw output lebih mudah diaudit dan diparse.

12. Timing & Noise

Nmap punya timing template -T0 sampai -T5 serta banyak kontrol rate/retry. Semakin agresif bukan berarti semakin benar. Pada jaringan lambat, packet loss dapat membuat hasil lebih buruk. Mulai konservatif, ukur, lalu naikan hanya jika scope dan kondisi mendukung.

- Default timing adalah baseline yang baik untuk banyak kasus.
- Jangan otomatis memakai -T5 pada production network.
- Parallelism dan rate tinggi bisa memicu alert atau membebani target.
- Catat perubahan timing di activity log karena memengaruhi repeatability.

13. Protocol-specific Enumeration Mindset

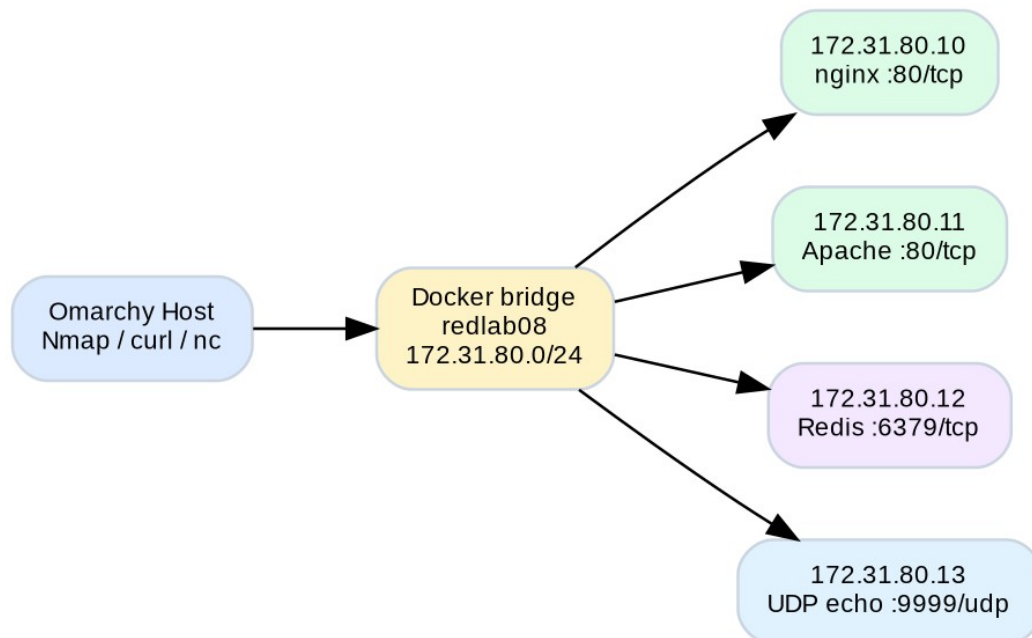
Setelah service teridentifikasi, pertanyaan berubah dari "port apa yang open?" menjadi "apa yang dapat dipelajari dari protocol ini tanpa melampaui scope?".

Service	Pertanyaan awal	Verifikasi aman
HTTP/HTTPS	Status, headers, title, redirect, TLS, virtual host?	curl -I / curl -v; browser/Burp pada target authorized.
SSH	Banner, protocol, host key, authentication methods?	Version detection; jangan brute force tanpa izin.
FTP	Banner, anonymous login diizinkan?	Gunakan explicit check hanya jika RoE mengizinkan.
SMB	Dialect, signing, shares, domain context?	Akan diperdalam pada Windows/AD modules.
DNS	Authoritative records, recursion, zone transfer policy?	dig; transfer hanya bila explicitly authorized.
SNMP	Version/community configuration?	UDP enumeration; credential guessing perlu izin eksplisit.
LDAP	Directory endpoint, TLS, anonymous bind policy?	Akan diperdalam pada AD modules.
Redis	Protocol reachable, authentication/protected mode?	PING/info hanya pada lab/authorized target.

Stop condition

Kalau suatu technique mulai mengubah data, menebak credential, membuat banyak request, atau berpotensi mengganggu availability, kembali ke RoE. Enumeration tidak berarti semua probe otomatis boleh.

14. Lab Topology



Four intentionally simple services on a private Docker bridge.

Lab script membuat network user-defined redlab08. Tidak ada -p/--publish ke interface host, sehingga service tidak sengaja dibuka ke LAN. Host Omarchy tetap dapat mengakses container bridge IP pada setup Docker Linux standar.

```
./labs/multi-service-lab.sh create
./labs/multi-service-lab.sh status
```

Setelah selesai

```
./labs/multi-service-lab.sh cleanup
```

15. Hands-on Lab 1 - Workspace & Scope Guardrail

```
./labs/lab-init.sh module08-enum
cat module08-enum/00-scope/README.md
```

Utility lab-nmap.py hanya menerima 127.0.0.0/8 dan 172.31.80.0/24. Guardrail ini bukan pengganti authorization, tetapi membantu mencegah typo target selama training.

Expected: ditolak

```
python labs/lab-nmap.py 192.168.1.1 -- -sn
```

Allowed training subnet

```
python labs/lab-nmap.py 172.31.80.0/28 -- -sn -n
```

16. Hands-on Lab 2 - Host Discovery

```
./labs/multi-service-lab.sh create
python labs/lab-nmap.py 172.31.80.0/28 -- -sn -n -oA module08-enum/01-discovery/hosts
```

Subnet /28 meliputi .0-.15, jadi gateway Docker dan container .10-.13 dapat muncul. Catat host mana yang up dan jangan menyimpulkan identitas service dari IP saja.

- [] Temukan minimal empat container training.
- [] Bedakan gateway bridge dari service container.
- [] Simpan output normal dan XML.

17. Hands-on Lab 3 - Open vs Closed TCP

```
python labs/lab-nmap.py 172.31.80.10 -- -n -p 80,65000
```

Port 80 harus open pada nginx. Port 65000 biasanya closed. Kalau hasil berbeda, gunakan docker status dan ss/docker inspect untuk troubleshooting sebelum mengubah scan flags.

18. Hands-on Lab 4 - SYN vs Connect Scan

```
nmap -sT -n -p 80 172.31.80.10
sudo nmap -sS -n -p 80 172.31.80.10
```

Bandingkan output, kebutuhan privilege, dan packet behavior. Jangan mengejar istilah "stealth"; fokus ke mekanisme dan telemetry.

19. Hands-on Lab 5 - Service/Version Detection

```
nmap -n -sV -p 80 172.31.80.10
nmap -n -sV -p 80 172.31.80.11
nmap -n -sV -p 6379 172.31.80.12
```

Tujuan: melihat bahwa dua host pada port 80 dapat menjalankan product berbeda, dan bahwa nomor port bukan satu-satunya sumber identifikasi.

Target	Expected family	Manual follow-up
172.31.80.10:80	nginx	curl -I http://172.31.80.10/
172.31.80.11:80	Apache httpd	curl -I http://172.31.80.11/
172.31.80.12:6379	Redis	RESP PING via nc

20. Hands-on Lab 6 - Manual Verification

```
./labs/manual-verify.sh
```

Bandingkan output manual dengan -sV. Kalau scanner menampilkan version tapi manual response tidak mendukung detail tersebut, catat confidence, bukan asumsi.

21. Hands-on Lab 7 - UDP

```
printf 'hello' | nc -u -w 2 172.31.80.13 9999
sudo nmap -sU -n -p 9999 172.31.80.13
```

UDP echo service akan mengembalikan marker REDLAB08. Manual packet exchange memberi bukti bahwa service memang menerima datagram, bukan hanya open|filtered inference.

22. Hands-on Lab 8 - Explicit NSE

```
nmap --script-help http-title
nmap -n -p 80 --script http-title,http-headers 172.31.80.10
nmap -n -p 6379 --script redis-info 172.31.80.12
```

Read before run

Gunakan --script-help sebelum script baru. Hindari kategori all, vuln, exploit, dos, brute, atau intrusive kecuali lab/engagement secara eksplisit memang membutuhkannya dan mengizinkan dampaknya.

23. Hands-on Lab 9 - Structured Evidence

```
nmap -n -sV -p 80,6379 -oA module08-enum/02-scans/selected 172.31.80.10 172.31.80.12
```

```
python labs/nmap-inventory.py \
  module08-enum/02-scans/selected.xml \
  --output module08-enum/04-inventory/services.csv
```

```
column -s, -t < module08-enum/04-inventory/services.csv | less -S
```

CSV adalah normalized inventory; file XML tetap dipertahankan sebagai evidence sumber. Jangan mengganti evidence mentah dengan hasil transformasi.

24. Hands-on Lab 10 - Hypothesis-driven Enumeration

Tanpa walkthrough, lakukan proses berikut pada keempat target lab dan tulis alasan tiap command di notes:

1. Host discovery pada /28.
2. Pilih port scan yang masuk akal untuk setiap host.
3. Jalankan -sV hanya pada port yang relevan.
4. Pilih maksimal dua NSE scripts eksplisit untuk HTTP/Redis.
5. Lakukan manual verification.
6. Update inventory CSV dan buat summary singkat.

Success criterion

Lo bisa menjelaskan dari mana setiap kesimpulan berasal: packet/scan output, service probe, NSE, atau manual protocol response.

25. Mini Project - Enumeration Case File

Buat deliverable lokal berikut:

```
module08-enum/
├── 00-scope/
├── 01-discovery/
├── 02-scans/
├── 03-manual/
├── 04-inventory/services.csv
├── 05-evidence/
└── 06-summary/enumeration-summary.md
```

Summary minimal harus mencantumkan target, open ports, service/product/version confidence, manual evidence, unknowns, dan next safe question.

26. Practical Challenge - 45 Menit

Mulai dari lab yang baru dibuat. Jangan melihat section hasil yang sudah lo tulis sebelumnya. Target challenge: buat inventory yang dapat direview orang lain.

- [] Buat workspace baru.
- [] Discover host pada 172.31.80.0/28.
- [] Identifikasi service keempat container.
- [] Gunakan minimal satu TCP SYN scan, satu TCP connect scan, dan satu UDP scan.
- [] Gunakan -sV pada port terpilih.
- [] Gunakan explicit NSE hanya setelah membaca --script-help.
- [] Manual verify semua service.
- [] Export -oA dan parse XML ke CSV.
- [] Tulis 5-10 baris summary tanpa mengklaim vulnerability yang belum dibuktikan.
- [] Cleanup Docker lab.

Tidak lulus kalau...

Lo hanya menjalankan nmap -A lalu menyalin output. Goal module ini adalah methodology dan reasoning, bukan satu command serba bisa.

27. Common Mistakes

Kesalahan	Kenapa bermasalah	Perbaiki
Scanning tanpa scope	Legal/operational risk	Validasi target list dan RoE dahulu.
Menganggap port number = service	Service bisa pindah port	Gunakan -sV + manual verification.
Langsung -A/-sC ke production	Menambah probes/script tanpa review	Pilih fitur eksplisit berdasarkan tujuan.
Mengabaikan UDP	Inventory dapat kehilangan service penting	Lakukan UDP hypothesis-driven.
Tidak menyimpan XML	Sulit diotomasi dan diaudit	Gunakan -oA / XML.
Percaya banner mentah	Banner bisa spoofed/hidden	Cross-check behavior.
Terlalu cepat menaikkan timing	False negative / disruption	Mulai konservatif dan ukur.

28. Knowledge Check

1. Kenapa state port Nmap bukan properti permanen?
2. Apa beda -sn dan -Pn?
3. Kapan -sT berguna dibanding -sS?
4. Kenapa service detection tidak cukup hanya dari nomor port?
5. Apa arti open|filtered pada UDP?
6. Kenapa -sC perlu diperlakukan hati-hati?
7. Apa keuntungan -oA dan XML?
8. Kenapa manual verification tetap wajib setelah -sV?
9. Kenapa -T5 bukan default terbaik?
10. Apa bedanya discovery, enumeration, dan vulnerability validation?

29. Exit Criteria

- [] Saya bisa menjelaskan enam Nmap port states.
- [] Saya bisa menjalankan host discovery dan port scan pada lab authorized.
- [] Saya paham perbedaan -sT, -sS, dan -sU.
- [] Saya bisa menggunakan -sV tanpa memperlakukan output sebagai kebenaran absolut.
- [] Saya bisa membaca --script-help dan memilih NSE script eksplisit.
- [] Saya bisa menyimpan -oA dan parse XML ke CSV.
- [] Saya bisa manual verify HTTP, Redis, dan UDP echo di lab.
- [] Saya memahami bahwa scanning tetap harus mengikuti scope dan RoE.

30. Cheat Sheet

```
# Host discovery
nmap -sn -n 172.31.80.0/28

# TCP connect / SYN
nmap -sT -n -p 80 172.31.80.10
sudo nmap -sS -n -p 80 172.31.80.10

# Service detection
nmap -sV -n -p 80,6379 TARGET

# UDP
sudo nmap -sU -n -p 9999 172.31.80.13

# Explicit NSE
nmap --script-help http-title
nmap -p 80 --script http-title,http-headers TARGET

# Evidence
nmap -sV -oA scan-name TARGET
python labs/nmap-inventory.py scan-name.xml --output services.csv
```

31. Referensi Utama

Nmap - Port Scanning Basics - Port states dan core scanning concepts.

<https://nmap.org/book/man-port-scanning-basics.html>

Nmap - Host Discovery - -sn, -Pn, discovery probes, dan local ARP behavior.

<https://nmap.org/book/man-host-discovery.html>

Nmap - Service and Version Detection - -sV dan service probes.

<https://nmap.org/book/man-version-detection.html>

Nmap - UDP Scan - Cara Nmap menginterpretasikan UDP responses.

<https://nmap.org/book/scan-methods-udp-scan.html>

Nmap - Scripting Engine - NSE categories, -sC, script selection, dan safety warning.

<https://nmap.org/book/man-nse.html>

Arch Linux - nmap package - Package resmi Arch untuk network discovery/security auditing.

https://archlinux.org/packages/extra/x86_64/nmap/

Docker - Bridge network driver - User-defined bridge networking untuk isolated local lab.

<https://docs.docker.com/engine/network/drivers/bridge/>

Next module

Module 09 - Vulnerability Assessment: CVE, CWE, CVSS, scanner output, false positives, version correlation, reachability, manual validation, prioritization, dan remediation.